

Classical Synchronization Problems in C++ and Go

CS3211 Parallel and Concurrent Programming

Outline

- Why study classical synchronization problems?
 - Recap
- Problem solutions in C++ and Go
 - Barrier
 - Dining philosophers
 - Barber shop

Motivation

- Classical synchronization problems model problems that we have nowadays in our computer systems

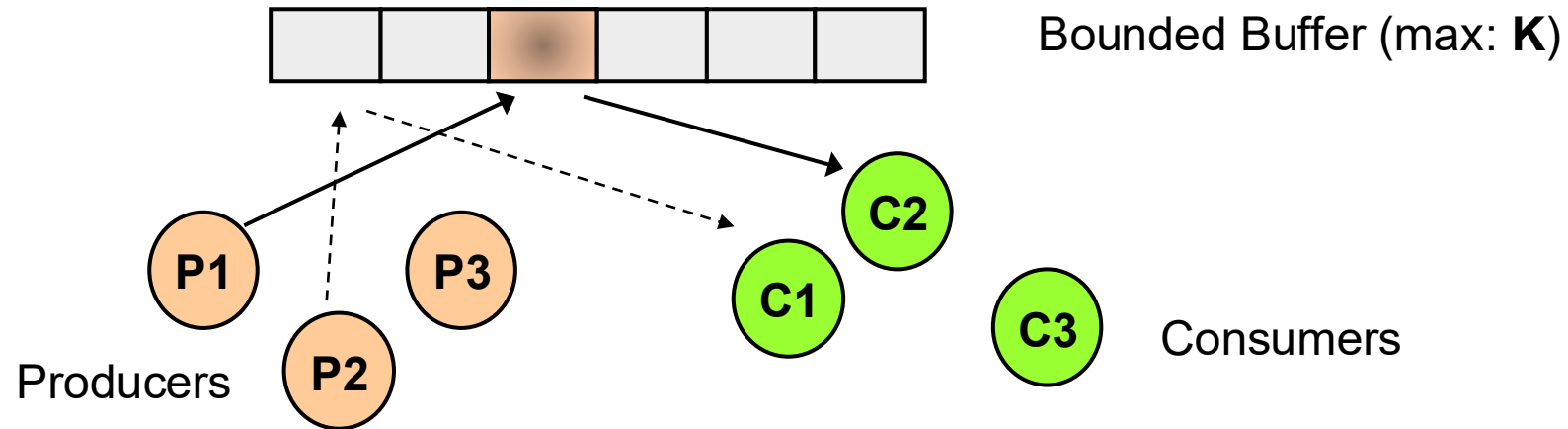
Problem	CS problem
Barrier	Wait until threads/processes reach a specific point in the execution
Producer-consumer	Model interactions between a processor and devices that interact through FIFO channels.
Readers-writers	Model access to shared memory
Dining philosophers	Allocation of limited resources to a group of processes in a deadlock-free and starvation-free manner.
Barbershop	Coordinating the execution of a processor.
FIFO Semaphore	Needed to avoid starvation and increase fairness in the system.
H2O	Allocation of specific resource to a process.
Cigarette smokers	The agent represents an operating system that allocates resources, and the smokers represent applications that need resources.

Recap from your OS class

- Producer consumer problem
- Readers writers problem

Producer consumer: specification

- Processes share a buffer (bounded or unbounded)
 - Producers produce items to insert in buffer
 - Only when the buffer is not full ($< K$ items)
 - Consumers remove items from buffer
 - Only when the buffer is not empty (> 0 items)



Producer consumer in tutorial 1

- <https://godbolt.org/z/66zsjWrhd>

Producer consumer: buffered version



Example Buffer (**K = 4**)

```
while (TRUE) {  
    Produce Item;  
  
    wait(notFull) ;  
    wait(mutex) ;  
    buffer[in] = item;  
    in = (in+1) % K;  
    count++;  
    signal(mutex) ;  
    signal(notEmpty) ;  
}
```

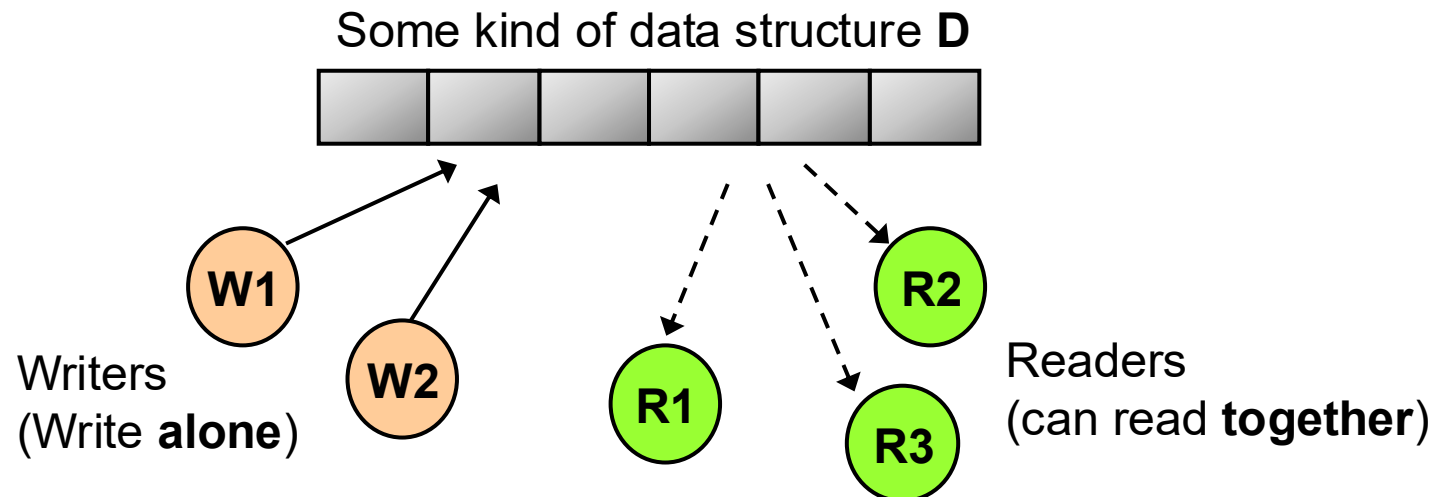
Producer

```
while (TRUE) {  
  
    wait(notEmpty) ;  
    wait(mutex) ;  
    item = buffer[out];  
    out = (out+1) % K;  
    count--;  
    signal(mutex) ;  
    signal(notFull) ;  
  
    Consume Item;  
}
```

Consumer

Readers writers: specification

- Processes share a data structure D:
 - Reader: retrieves information from D
 - Writer: modifies information in D
- Writer must have exclusive access to D
- Reader can access with other readers



Readers writers

Writers

```
roomEmpty.wait ()  
    #critical section for writers  
roomEmpty.signal ()
```

Start with:

```
counter = 0  
mutex = Semaphore (1)  
roomEmpty = Semaphore (1)
```

Readers

```
mutex.wait ()  
readers += 1  
if readers == 1:  
    roomEmpty.wait () # first in locks  
mutex.signal ()  
# critical section for readers  
mutex.wait ()  
readers -= 1  
if readers == 0:  
    roomEmpty.signal () # last out unlocks  
mutex.signal ()
```

Lightswitch definition

Lightswitch :

```
counter = 0
```

```
mutex = Semaphore (1)
```

```
lock (semaphore):
```

```
    mutex.wait ()
```

```
    counter += 1
```

```
    if counter == 1:
```

```
        semaphore.wait ()
```

```
    mutex.signal ()
```

```
unlock (semaphore):
```

```
    mutex.wait ()
```

```
    counter -= 1
```

```
    if counter == 0:
```

```
        semaphore.signal ()
```

```
    mutex.signal ()
```

Readers

```
mutex.wait ()
```

```
readers += 1
```

```
if readers == 1:
```

```
    roomEmpty.wait () # first in locks
```

```
mutex.signal ()
```

```
# critical section for readers
```

```
mutex.wait ()
```

```
readers -= 1
```

```
if readers == 0:
```

```
    roomEmpty.signal () # last out unlocks
```

```
mutex.signal ()
```

Readers writers with light switch

Writers

```
roomEmpty.wait ()  
    #critical section for writers  
roomEmpty.signal ()
```

Readers

```
readLightswitch.lock(roomEmpty)  
    # critical section  
readLightswitch.unlock(roomEmpty)
```

Starvation of writers is possible

#starving writers

Use a

turnstile = Semaphore (1)

No-starve readers writers

Writers

turnstile.wait ()

roomEmpty.wait ()

critical section for writers

turnstile.signal ()

roomEmpty.signal ()

Readers

turnstile.wait ()

turnstile.signal ()

readSwitch.lock (roomEmpty)

critical section for readers

readSwitch.unlock (roomEmpty)

Readers-writers locks

- Programming languages have rwlocks
 - C++17 onwards: use `shared_mutex`
 - Go has `RWLock`

```
6 class ThreadSafeCounter {
7     public:
8         ThreadSafeCounter() = default;
9
10        // Multiple threads/readers can read
11        // the counter's value at the same time.
12        unsigned int get() const {
13            std::shared_lock lock(mutex_);
14            return value_;
15        }
16
17        // Only one thread/writer can
18        // increment/write the counter's value.
19        unsigned int increment() {
20            std::unique_lock lock(mutex_);
21            return ++value_;
22        }
23
24        // Only one thread/writer can reset/write
25        // the counter's value.
26        void reset() {
27            std::unique_lock lock(mutex_);
28            value_ = 0;
29        }
30
31    private:
32        mutable std::shared_mutex mutex_;
33        unsigned int value_ = 0;
34    };

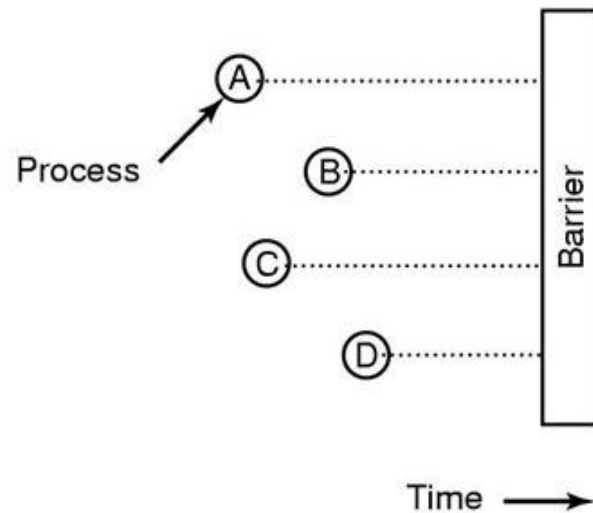
```

Outline

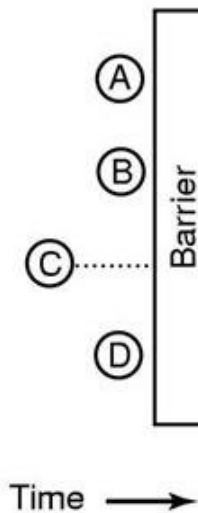
- Why study classical synchronization problems?
- Problem solutions in C++ and Go
 - Barrier
 - Dining philosophers
 - Barber shop

Barrier: specification

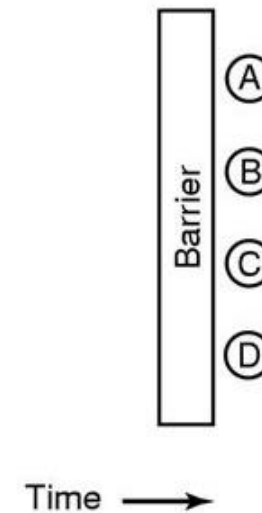
- Any thread/process must stop at this point and cannot proceed until all other threads/processes reach this barrier



(a)



(b)



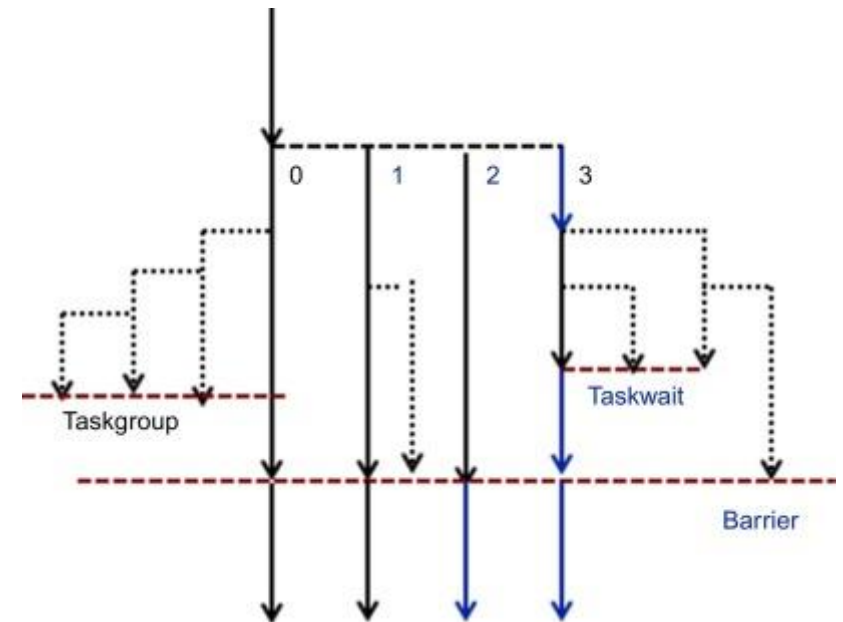
(c)

Types of barriers

- Single use barrier or latch (`std::latch` in C++20)
 - Starts in the raised state and cannot be re-raised once it is in the lowered state
- Reusable barriers (`std::barrier` in C++20)
 - Once the arriving threads are unblocked from a barrier phase's synchronization point, the same barrier can be reused
 - Combining tree barrier - a hierarchical way of implementing barrier
 - Resolve the scalability by avoiding the case that all threads are spinning at the same location

Barrier usage

- Appears in many collective routines as part of directive-based parallel languages
 - Parallel for loop in OpenMP
 - Collective communication in MPI
- Part of the programming language
 - `std::barrier` in C++20
 - `WaitGroup wait` in Go



C++: std::barrier

- In C++, create a synchronization point
 - Lines 21 & 25: wait for all threads to arrive at the barrier

```
7 int main() {
8     const auto tas = { "hw", "g", "zr", "c" };
9
10    auto on_completion = []() noexcept {
11        // locking not needed here
12        static auto phase = "... done\n" "Cleaning up...\n";
13        std::cout << phase;
14        phase = "... done\n";
15    };
16    std::barrier sync_point(std::ssize(tas), on_completion);
17
18    auto work = [&](std::string name) {
19        std::string product = " " + name + " worked\n";
20        std::cout << product;
21        sync_point.arrive_and_wait();
22
23        product = " " + name + " cleaned\n";
24        std::cout << product;
25        sync_point.arrive_and_wait();
26    };
27
28    std::cout << "Starting...\n";
29    std::vector<std::thread> threads;
30    for (auto const& worker : tas) {
31        threads.emplace_back(work, worker);
32    }
33    for (auto& thread : threads) {
34        thread.join();
35    }
36 }
```

C++: Barrier implementation

- Attempt 1:
 - Threads can go ahead other threads by one lap

```
7 struct BarrierAttempt1 {
8     std::ptrdiff_t expected;
9     std::ptrdiff_t count;
10    std::mutex mut;
11    std::counting_semaphore<> turnstile;

12
13    BarrierAttempt1(std::ptrdiff_t expected)
14        : expected{expected}, count{0}, mut{}, turnstile{0} {}
15
16    void arrive_and_wait() {
17    {
18        std::scoped_lock lock{mut};
19        count++;
20        if (count == expected) {
21            // Open turnstile
22            turnstile.release();
23        }
24    }

25
26    turnstile.acquire();
27    turnstile.release();
28
29    {
30        std::scoped_lock lock{mut};
31        count--;
32        if (count == 0) {
33            // Close turnstile to reset barrier
34            turnstile.acquire();
35        }
36    }
37 }
38 };
```

C++: Barrier implementation (2)

```
40 struct Barrier2 {
41     std::ptrdiff_t expected;
42     std::ptrdiff_t count;
43     std::mutex mut;
44     std::counting_semaphore<> turnstile;
45     std::counting_semaphore<> turnstile2;
46
47     Barrier2(std::ptrdiff_t expected)
48         : expected{expected}, count{0}, mut{},
49         |turnstile{0}, turnstile2{1} {}
50
51     void arrive_and_wait() {
52     {
53         std::scoped_lock lock{mut};
54         count++;
55         if (count == expected) {
56             // Close waiter turnstile
57             turnstile2.acquire();
58             // Open turnstile into the critical section
59             turnstile.release();
60         }
61     }
62
63     turnstile.acquire();
64     turnstile.release();
```

- Use 2 turnstiles: raise and lower the barrier

```
40 struct Barrier2 {
41     std::ptrdiff_t expected;
42     std::ptrdiff_t count;
43     std::mutex mut;
44     std::counting_semaphore<> turnstile;
45     std::counting_semaphore<> turnstile2;
46
47     Barrier2(std::ptrdiff_t expected)
48         : expected{expected}, count{0}, mut{},
49         |turnstile{0}, turnstile2{1} {}
50
51     void arrive_and_wait() {
52     {
53         std::scoped_lock lock{mut};
54         count++;
55         if (count == expected) {
56             // Close waiter turnstile
57             turnstile2.acquire();
58             // Open turnstile into the critical section
59             turnstile.release();
60         }
61     }
62
63     turnstile.acquire();
64     turnstile.release();
65
66     {
67         std::scoped_lock lock{mut};
68         count--;
69         if (count == 0) {
70             // Close turnstile to reset barrier
71             turnstile.acquire();
72             // Open second turnstile to let waiters through
73             turnstile2.release();
74         }
75     }
76
77     turnstile2.acquire();
78     turnstile2.release();
79 }
80 };
```

C++: Barrier implementation (3)

```
82 // Use a preloaded turnstile to let threads through faster
83 struct Barrier3 {
84     std::ptrdiff_t expected;
85     std::ptrdiff_t count;
86     std::mutex mut;
87     std::counting_semaphore<> turnstile;
88     std::counting_semaphore<> turnstile2;
89
90     Barrier3(std::ptrdiff_t expected)
91         : expected{expected}, count{0}, mut{},
92         |turnstile{0}, turnstile2{1} {}
93
94     void arrive_and_wait() {
95     {
96         std::scoped_lock lock{mut};
97         count++;
98         if (count == expected) {
99             // Close waiter turnstile
100             turnstile2.acquire();
101             // Open turnstile into the critical section
102             turnstile.release(expected);
103         }
104     }
105
106     turnstile.acquire();
107 }
```

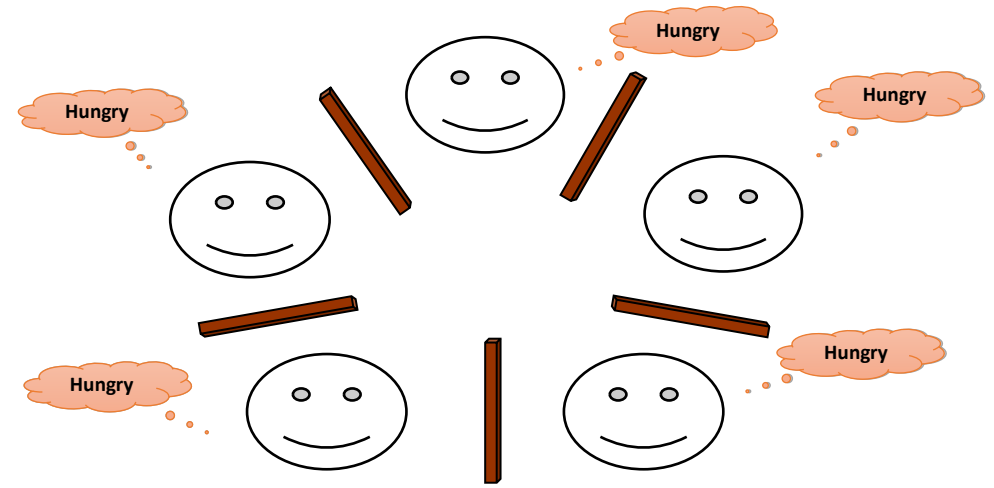
- Lines 102 and 115: counting semaphore can be increased by expected to allow threads to pass

```
108 {
109     std::scoped_lock lock{mut};
110     count--;
111     if (count == 0) {
112         // Close turnstile to reset barrier
113         turnstile.acquire();
114         // Open second turnstile to let waiters through
115         turnstile2.release(expected);
116     }
117 }
118
119     turnstile2.acquire();
120 }
121 };
```

Go: Barrier implementation

- Use 2 WaitGroups for a reusable barrier for a group of goroutines

```
5 type Barrier1 struct {
6     wg  sync.WaitGroup
7     wg2 sync.WaitGroup
8 }
9
10 func (b *Barrier1) Init(expected int) {
11     b.wg.Add(expected)
12     b.wg2.Add(expected)
13 }
14
15 func (b *Barrier1) Wait() {
16     b.wg.Done()
17     b.wg.Wait()
18     // This line only reached when expected
19     // threads have called Wait
20     // Reset the barrier now
21     b.wg.Add(1)
22     b.wg2.Done()
23     // Wait because the barrier might not
24     // be fully reset yet
25     b.wg2.Wait()
26     // Now barrier is fully reset
27     b.wg2.Add(1)
28 }
```

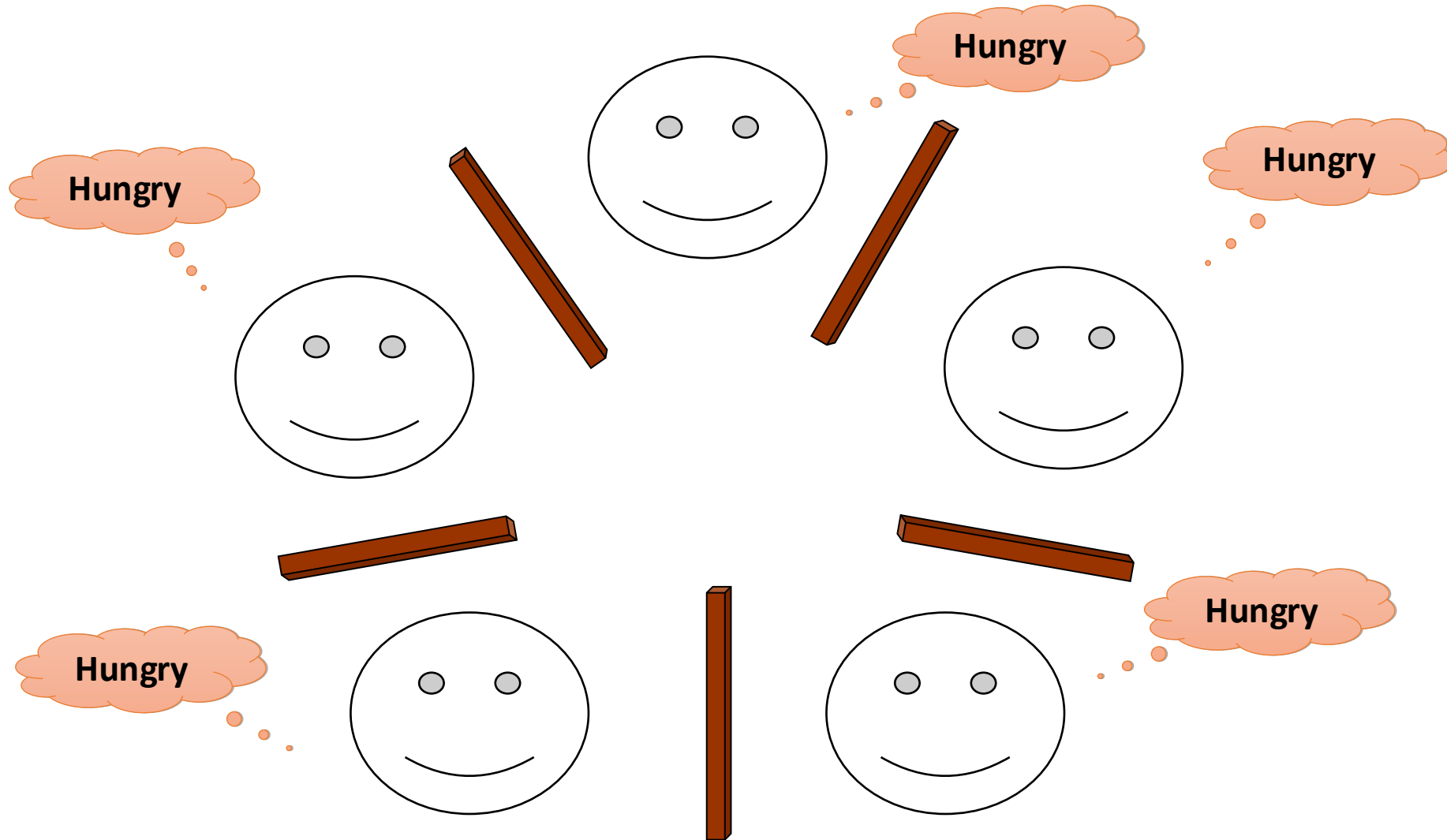


Dining Philosophers

Dining philosophers: specification

- Appeared when computers were competing for access to tape drive peripherals
- Models the problem of allocating limited resources to a group of processes in a deadlock-free and starvation-free manner
- An algorithm that solves this problem without deadlock
 - Low contention: performs wonderfully when the philosophers spend any appreciable amount of time thinking, compared to eating
 - High contention: philosophers are hungry

Dining philosophers: specification



Dining philosophers: attempt 1

```
#define N 5
#define LEFT i
#define RIGHT ((i+1) % N)

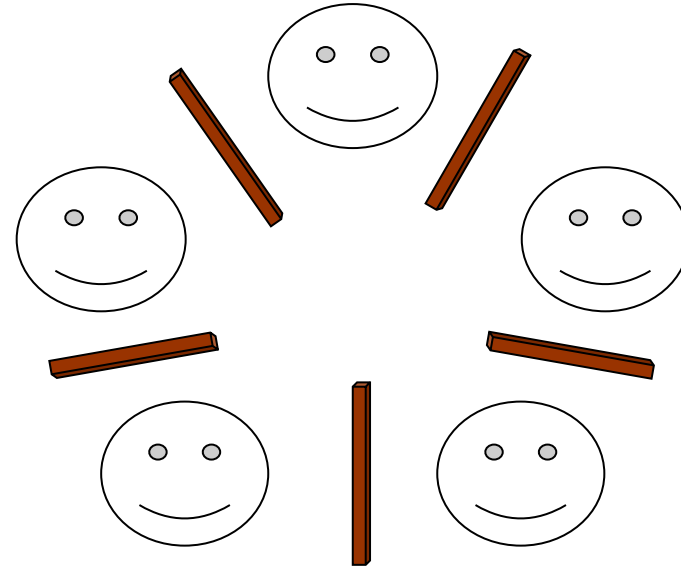
//For philosopher i
while (TRUE){

    Think();
    //hungry, need food!
    takeChpStick(LEFT);
    takeChpStick(RIGHT);

    Eat();

    putChpStick(LEFT);
    putChpStick(RIGHT);

}
```



Dining philosophers: attempt 1

- Deadlock:
 - All philosopher simultaneously takes up the left chopstick, and none can proceed
- Fix the attempt:
 - Make the philosopher to put down the left chopstick if right chopstick cannot be acquired
 - Try again later
 - No deadlock:
 - Livelock: All philosopher take up left chopstick, put it down, take it up, put it down,...

Dining philosophers: attempt 2

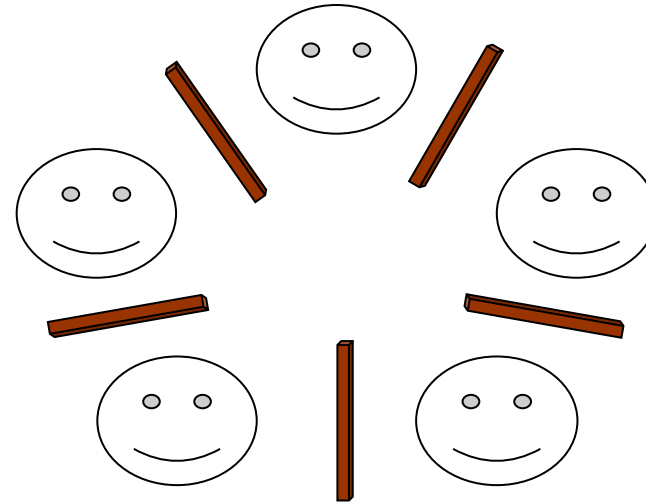
```
#define N 5
#define LEFT i
#define RIGHT ((i+1) % N)

//For philosopher i
while (TRUE){
    Think( );

    wait(mutex);

    takeChpStick(LEFT);
    takeChpStick(RIGHT);
    Eat( );
    putChpStick(LEFT);
    putChpStick(RIGHT);

    signal(mutex);
}
```



C++: Dining philosophers' implementation (1)

- Uses **some** deadlock avoidance algorithm
 - The objects are locked by an unspecified series of calls to lock, try_lock, and unlock. If a call to lock or unlock results in an exception, unlock is called for any locked objects before rethrowing
- https://howardhinnant.github.io/dining_philosophers.html

```
7 // Basic solution to the dining philosopher's problem
8 template <size_t NumP>
9 struct DiningTable1 {
10     using ChpStick = std::mutex;
11
12     ChpStick chpSticks[NumP];
13
14     // pid = philosopher id
15     ChpStick& get_left_chpStick(size_t pid) { return chpSticks[pid]; }
16     ChpStick& get_right_chpStick(size_t pid) { return chpSticks[(pid + 1) % NumP]; }
17
18     void eat(size_t pid, void (*eat_callback)(size_t pid)) {
19         std::scoped_lock lock{get_left_chpStick(pid), get_right_chpStick(pid)};
20         eat_callback(pid);
21     }
22 };
23
```

Go: Dining philosophers' implementation

- Use odd-even ring communication to avoid the deadlock

```
3 type ChpStick struct{}
4
5 type DiningTable1 struct {
6     numPhilosophers int
7     chpStickChs      []chan ChpStick
8 }
9
10 func (t *DiningTable1) Init(numPhilosophers int) {
11     t.numPhilosophers = numPhilosophers
12
13     t.chpStickChs = make([]chan ChpStick, 0, numPhilosophers)
14     for i := 0; i < numPhilosophers; i++ {
15         chpStick := make(chan ChpStick, 1)
16         chpStick <- ChpStick{}
17         t.chpStickChs = append(t.chpStickChs, chpStick)
18     }
19 }
42 func (t *DiningTable1) Eat(pid int, eat_callback func(pid int)) {
43     evenChpStickCh := t.evenChpStickCh(pid)
44     oddChpStickCh := t.oddChpStickCh(pid)
45
46     // Use even / odd chpSticks so the resulting
47     // chpStick locking order is acyclic
48     <-evenChpStickCh
49     <-oddChpStickCh
50
51     eat_callback(pid)
52
53     evenChpStickCh <- ChpStick{}
54     oddChpStickCh <- ChpStick{}
}
```

Go: Dining philosophers' implementation

- Use odd-even ring communication to avoid the deadlock

```
21 func (t *DiningTable1) leftChpStickCh(pid int) chan ChpStick {
22     return t.chpStickChs[pid]
23 }
24 func (t *DiningTable1) rightChpStickCh(pid int) chan ChpStick {
25     return t.chpStickChs[(pid+1)%t.numPhilosophers]
26 }
27 func (t *DiningTable1) evenChpStickCh(pid int) chan ChpStick {
28     if pid%2 == 0 {
29         return t.leftChpStickCh(pid)
30     } else {
31         return t.rightChpStickCh(pid)
32     }
33 }
34 func (t *DiningTable1) oddChpStickCh(pid int) chan ChpStick {
35     if pid%2 == 1 {
36         return t.leftChpStickCh(pid)
37     } else {
38         return t.rightChpStickCh(pid)
39     }
40 }
```

```
42 func (t *DiningTable1) Eat(pid int, eat_callback func(pid int
43     evenChpStickCh := t.evenChpStickCh(pid)
44     oddChpStickCh := t.oddChpStickCh(pid)
45
46     // Use even / odd chpSticks so the resulting
47     // chpStick locking order is acyclic
48     <-evenChpStickCh
49     <-oddChpStickCh
50
51     eat_callback(pid)
52
53     evenChpStickCh <- ChpStick{}
54     oddChpStickCh <- ChpStick{}
```

Go: Dining philosophers' implementation (2)

- Swap order of chopsticks to avoid deadlock

```
62 func (t *DiningTable2) Init(numPhilosophers int) {
63     t.numPhilosophers = numPhilosophers
64
65     t.chpStickChs = make([]chan ChpStick, 0, numPhilosophers)
66     for i := 0; i < numPhilosophers; i++ {
67         chpStick := make(chan ChpStick, 1)
68         chpStick <- ChpStick{}
69         t.chpStickChs = append(t.chpStickChs, chpStick)
70     }
71 }
```

```
94 func (t *DiningTable2) Eat(pid int, eat_callback func(pid int)) {
95     evenChpStickCh := t.evenChpStickCh(pid)
96     oddChpStickCh := t.oddChpStickCh(pid)
97
98     |
99     breakLock:
100     for {
101         <-evenChpStickCh
102         select {
103             case <-oddChpStickCh:
104                 break breakLock
105             default:
106                 // Couldn't get oddChpStickCh, swap order of chpSticks
107             }
108             evenChpStickCh <- ChpStick{}
109
110             <-oddChpStickCh
111             select {
112                 case <-evenChpStickCh:
113                     break breakLock
114                 default:
115                     // Couldn't get evenChpStickCh, swap order of chpSticks
116                 }
117             oddChpStickCh <- ChpStick{}
118         }
119
120         eat_callback(pid)
121
122         evenChpStickCh <- ChpStick{}
123         oddChpStickCh <- ChpStick{}
124     }
```

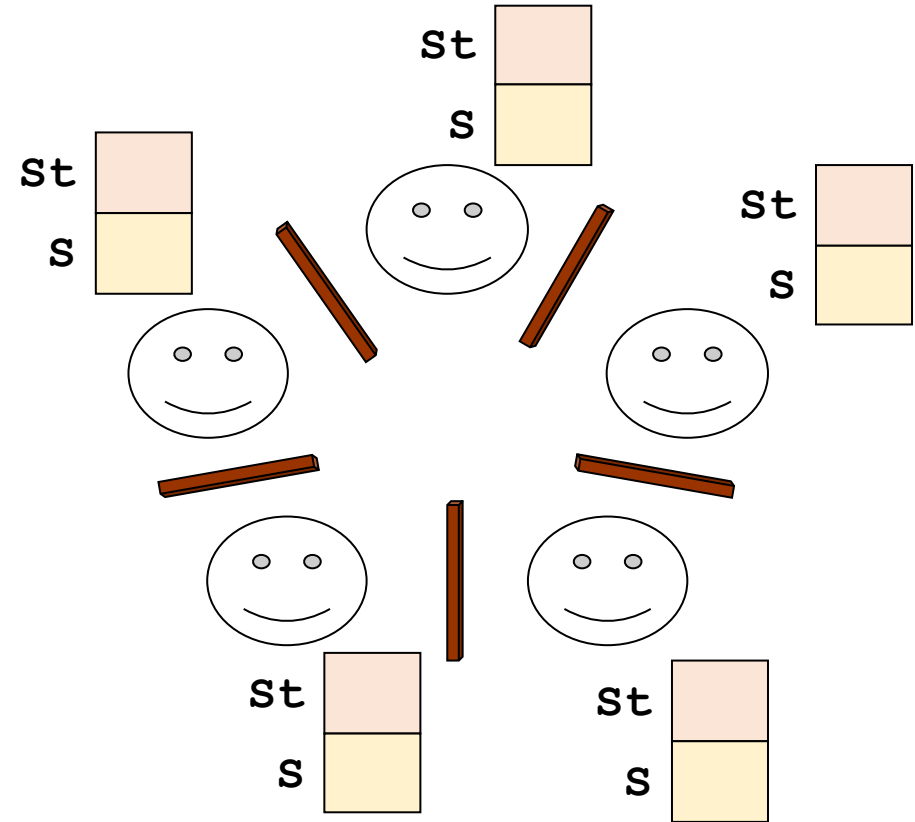
Dining philosophers: Tanenbaum solution

```
#define N 5
#define LEFT ((i+N-1)% N)
#define RIGHT ((i+1) % N)

#define THINKING 0
#define HUNGRY 1
#define EATING 2

int state[N];
Semaphore mutex = 1;
Semaphore s[N];

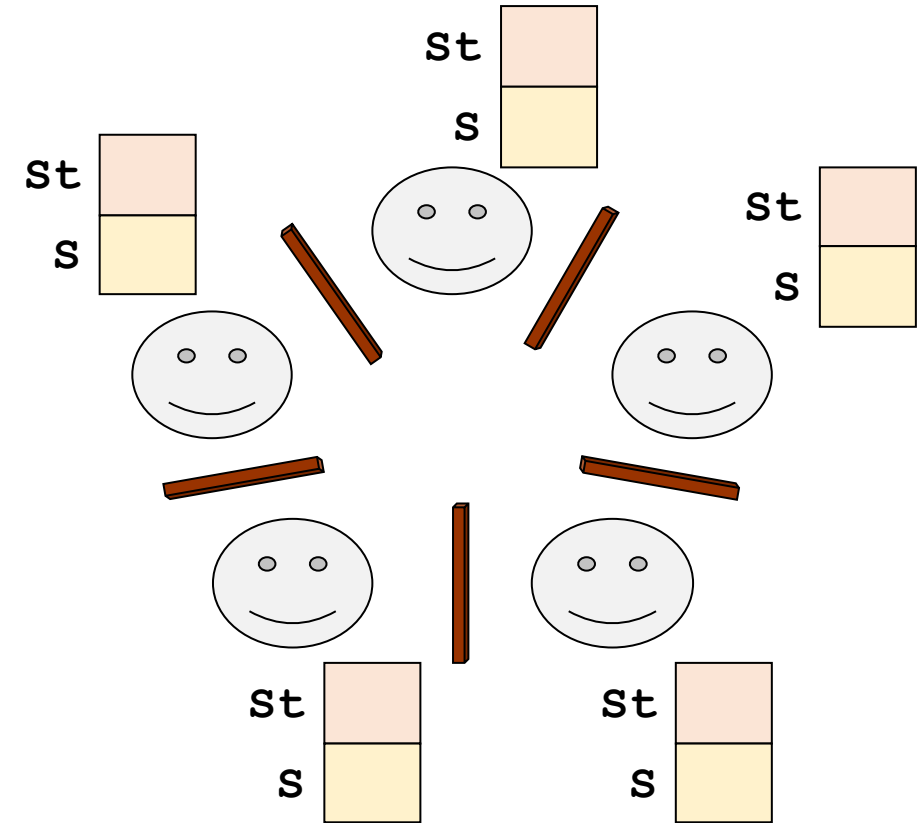
void philosopher(int i){
    while (TRUE){
        Think();
        takeChpSticks(i);
        Eat();
        putChpSticks(i);
    }
}
```



Dining philosophers: Tanenbaum solution

```
void takeChpSticks( i )  
{  
    wait(mutex);  
    state[i] = HUNGRY;  
    safeToEat(i);  
    signal(mutex);  
    wait(s[i]);  
}
```

```
void safeToEat( i )  
{  
    if( (state[i] == HUNGRY) &&  
        (state[LEFT] != EATING) &&  
        (state[RIGHT] != EATING) ) {  
        state[i] = EATING;  
        signal(s[i]);  
    }  
}
```



Dining philosophers: Tanenbaum solution

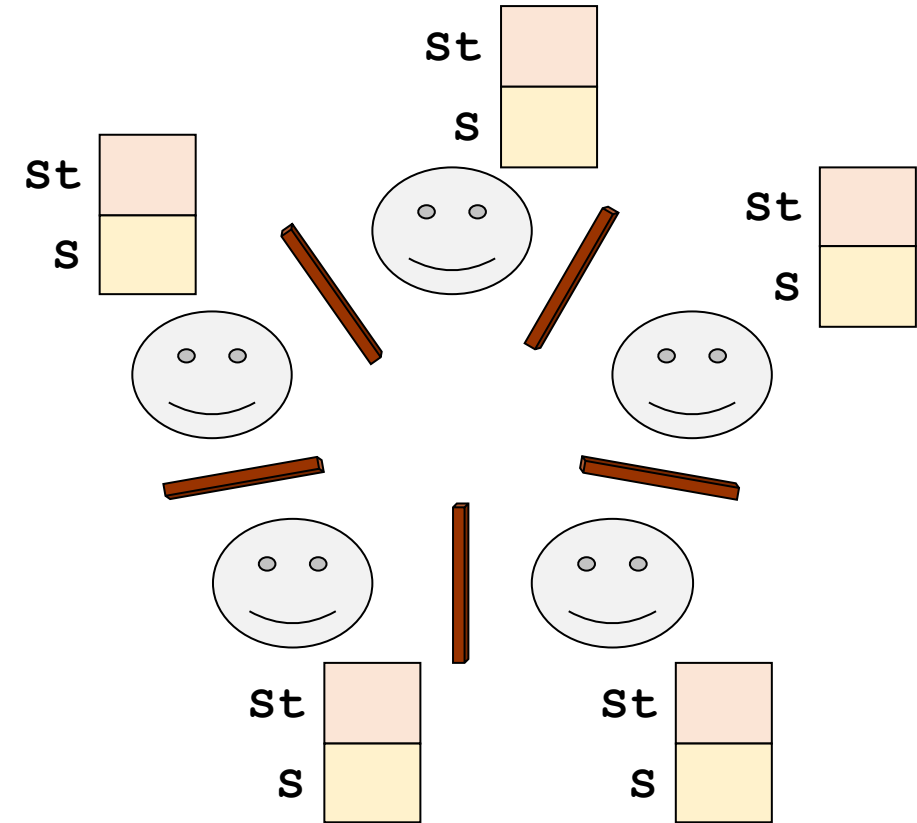
```
void safeToEat(i)
{
    if( (state[i] == HUNGRY) &&
        (state[LEFT] != EATING) &&
        (state[RIGHT] != EATING) ) {

        state[i] = EATING;
        signal(s[i]);
    }
}
```

```
void putChpSticks(i)
{
    wait(mutex);

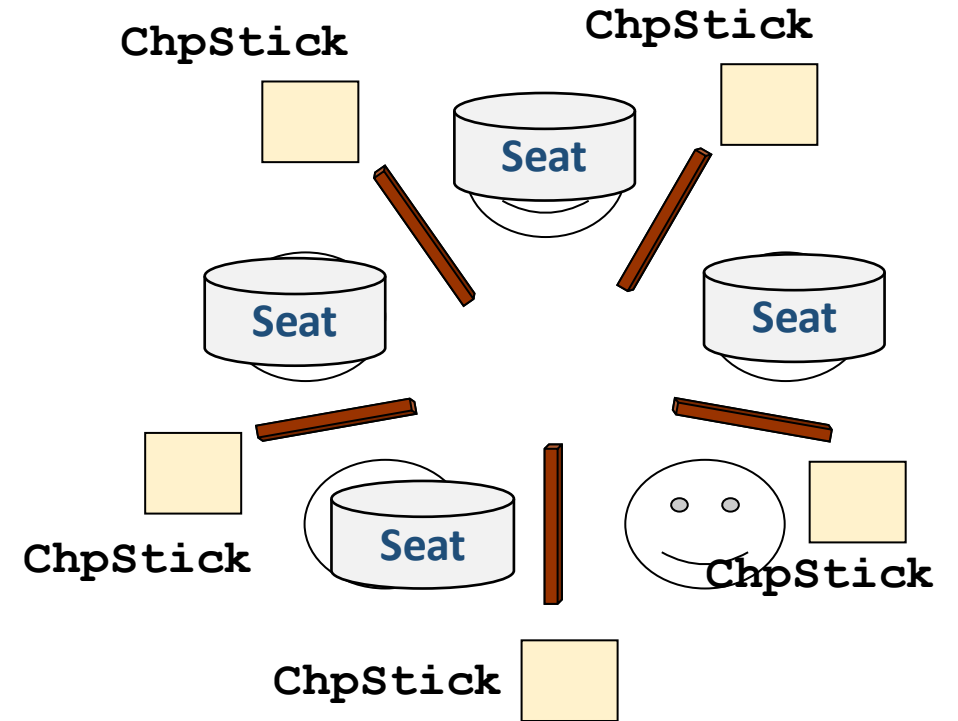
    state[i] = THINKING;
    safeToEat(LEFT);
    safeToEat(RIGHT);

    signal(mutex);
}
```



Dining philosophers: limited eater

```
void philosopher(int i){  
  
    while (TRUE){  
        Think( );  
        wait(seats);  
        wait(chpStick[LEFT]);  
        wait(chpStick[RIGHT]);  
        Eat( );  
        signal(chpStick[LEFT]);  
        signal(chpStick[RIGHT]);  
        signal(seats);  
    }  
}
```



C++: Dining philosophers' implementation (2)

- `footman_sem` used to limit the number of eaters
- Starvation is possible because mutexes and semaphores are not fair in C++

```
30     template <size_t NumP>
31     struct DiningTable2 {
32         using ChpStick = std::mutex;
33
34         ChpStick chpSticks[NumP];
35         std::counting_semaphore<> footman_sem{NumP - 1};
36
37         // pid = philosopher id
38         ChpStick& get_left_chpStick(size_t pid) { return chpSticks[pid]; }
39         ChpStick& get_right_chpStick(size_t pid) { return chpSticks[(pid + 1) % NumP]; }
40
41         void eat(size_t pid, void (*eat_callback)(size_t pid)) {
42             footman_sem.acquire();
43             std::scoped_lock left_lock{get_left_chpStick(pid)};
44             std::scoped_lock right_lock{get_right_chpStick(pid)};
45             eat_callback(pid);
46             footman_sem.release();
47         }
48     };

```

C++: Dining philosophers' implementation (3)

- Use a fair semaphore
 - FIFO semaphore will be discussed in tutorial 7

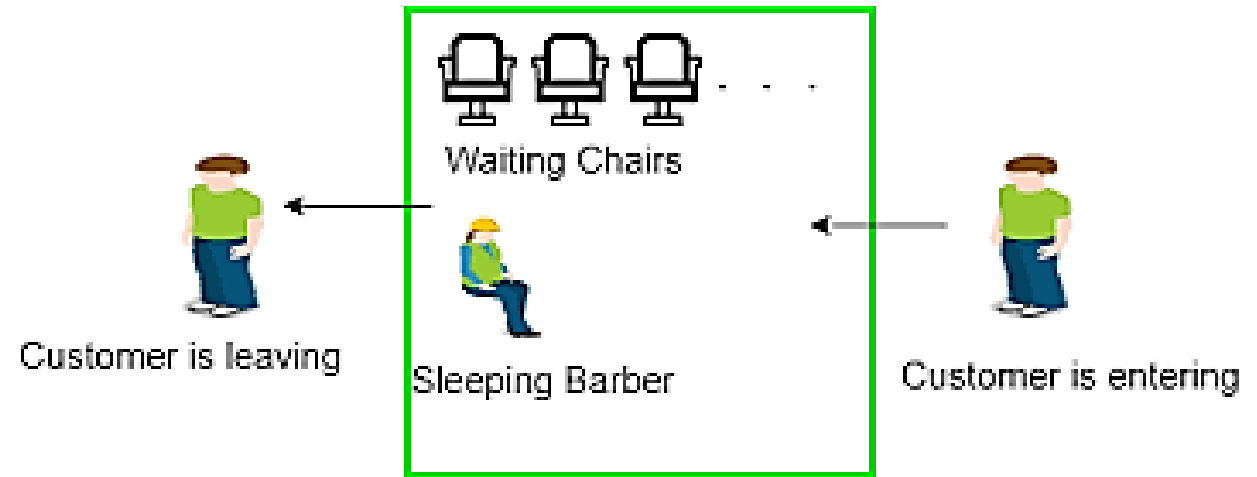
```
69     template <size_t NumP>
70     struct DiningTable3 {
71         using ChpStick = FairMutex;
72
73         ChpStick chpSticks[NumP];
74         FairSemaphore footman_sem{NumP - 1};
75
76         // pid = philosopher id
77         ChpStick& get_left_chpStick(size_t pid) { return chpSticks[pid]; }
78         ChpStick& get_right_chpStick(size_t pid) { return chpSticks[(pid + 1) % NumP]; }
79
80         void eat(size_t pid, void (*eat_callback)(size_t pid)) {
81             footman_sem.acquire();
82             std::scoped_lock left_lock{get_left_chpStick(pid)};
83             std::scoped_lock right_lock{get_right_chpStick(pid)};
84             eat_callback(pid);
85             footman_sem.release();
86         }
87     };
```

Go: Dining philosophers' implementation (3)

- This time it doesn't matter what order each philosopher tries to obtain each chopstick

```
126 type DiningTable3 struct {
127     numPhilosophers int
128     chpStickChs      []chan ChpStick
129     footman          chan struct{}
130 }
131
132 func (t *DiningTable3) Init(numPhilosophers int) {
133     t.numPhilosophers = numPhilosophers
134
135     t.chpStickChs = make([]chan ChpStick, 0, numPhilosophers)
136     for i := 0; i < numPhilosophers; i++ {
137         chpStick := make(chan ChpStick, 1)
138         chpStick <- ChpStick{}
139         t.chpStickChs = append(t.chpStickChs, chpStick)
140     }
141
142     t.footman = make(chan struct{}, numPhilosophers-1)
143     for i := 0; i < numPhilosophers-1; i++ {
144         t.footman <- struct{}{}
145     }
146 }
```

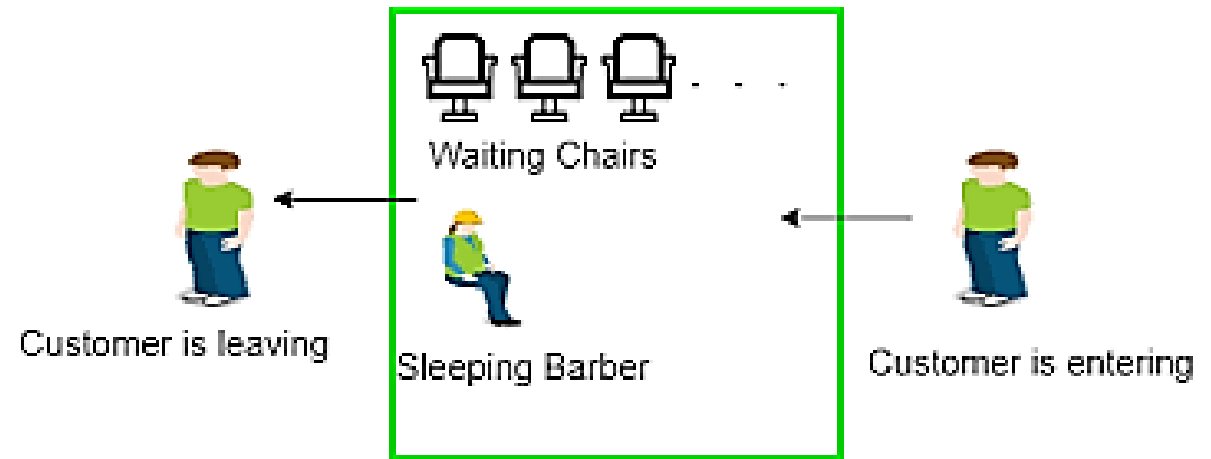
```
155 func (t *DiningTable3) Eat(pid int,
156     ..... eat_callback func(pid int)) {
157     leftChpStickCh := t.leftChpStickCh(pid)
158     rightChpStickCh := t.rightChpStickCh(pid)
159
160     <-t.footman
161
162     select {
163     case <-leftChpStickCh:
164         ..... <-rightChpStickCh
165     case <-rightChpStickCh:
166         ..... <-leftChpStickCh
167     }
168
169     eat_callback(pid)
170
171     leftChpStickCh <- ChpStick{}
172     rightChpStickCh <- ChpStick{}
173     t.footman <- struct{}{}
174 }
```



Barbershop

(Sleeping Barber)

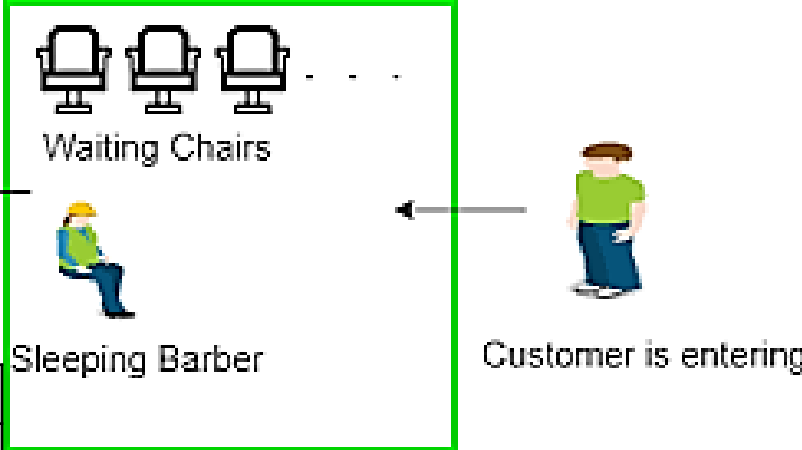
Problem description



- A barbershop consists of a waiting room with n chairs and the barber chair
- If there are no customers to be served, the barber goes to sleep
- If the barber is busy, but chairs are available, then the customer sits in one of the free chairs. If the barber is asleep, the customer wakes up the barber.
- If a customer enters the barbershop and all chairs are occupied, then the customer leaves the shop.

Solution

Customer is leaving



	Customer Pseudo-code		Barber Pseudo-code
1	<code>wait(mutex);</code>	21	
2	<code>if (customers == n) {</code>	22	
3	<code>signal(mutex);</code>	23	
4	<code>exit();</code>	24	
5	<code>}</code>	25	
6	<code>customers += 1;</code>	26	
7	<code>signal(mutex);</code>	27	<code>while (TRUE) {</code>
8	<code>signal(customer);</code>	28	<code>wait(customer);</code>
9	<code>wait(barber);</code>	29	<code>signal(barber);</code>
10	<code>getHairCut ();</code>	30	<code>cutHair();</code>
11	<code>signal(customerDone);</code>	31	<code>wait(customerDone);</code>
12	<code>wait (barberDone);</code>	32	<code>signal(barberDone);</code>
13	<code>wait(mutex);</code>	33	<code>}</code>
14	<code>customers -= 1;</code>	34	
15	<code>signal(mutex);</code>	35	

Line#	Initialization
1	<code>customers = 0</code>
2	<code>mutex = Semaphore (1)</code>
3	<code>customer = Semaphore (0)</code>
4	<code>barber = Semaphore (0)</code>
5	<code>customerDone = Semaphore (0)</code>
6	<code>barberDone = Semaphore (0)</code>

C++: Barbershop implementation

```
23 void customer(void (*balk)(), void (*getHairCut)()) {
24     {
25         std::scoped_lock lock{mut};
26         if (customers == MaxCustomers + 1) {
27             balk();
28             return;
29         }
30         customers++;
31     }
32
33     customer_sem.release();
34     barber_sem.acquire();
35     getHairCut();
36     done_customer_sem.release();
37     done_barber_sem.acquire();
38
39     {
40         std::scoped_lock lock{mut};
41         customers--;
42     }
43 }
44 };
```

```
4 template <size_t MaxCustomers>
5 struct Barbershop1 {
6     size_t customers;
7     std::mutex mut;
8     std::counting_semaphore<> customer_sem;
9     std::counting_semaphore<> barber_sem;
10    std::counting_semaphore<> done_customer_sem;
11    std::counting_semaphore<> done_barber_sem;
12
13    void barber(void (*cutHair)()) {
14        while (true) {
15            customer_sem.acquire();
16            barber_sem.release();
17            cutHair();
18            done_customer_sem.acquire();
19            done_barber_sem.release();
20        }
21    }
```

Go: Barbershop implementation

- Using channels
 - Customers might not get served in the FIFO order

```
23 func (bs *Barbershop1) Customer(balk func(), getHairCut func()) {
24     select {
25     case bs.chairs <- 1:
26         // Sat in chair, ask for barber
27         <-bs.barberAvailable
28         // Get hair cut
29         getHairCut()
30     default:
31         // Chairs are full
32         balk()
33     }
34 }
```

```
3 type Barbershop1 struct {
4     chairs          chan int
5     barberAvailable chan struct{}
6 }
7
8 func (bs *Barbershop1) Init(numChairs int) {
9     bs.chairs = make(chan int, numChairs)
10    bs.barberAvailable = make(chan struct{})
11 }
12
13 func (bs *Barbershop1) Barber(cutHair func()) {
14     for {
15         bs.barberAvailable <- struct{}{}
16         // Someone is sitting in the chair, free them
17         <-bs.chairs
18         // Cut their hair
19         cutHair()
20     }
21 }
```

Variations

- FIFO barbershop
- Multiple barbers

Summary

- Take advantage of the specification of the programming language when possible
- No fair semaphore
- Deadlock avoidance in
 - Acquiring multiple locks in C++
 - Odd-even ring communication

References

- The little book of semaphores:
<https://greenteapress.com/semaphores/LittleBookOfSemaphores.pdf>
- Dining philosophers:
https://howardhinnant.github.io/dining_philosophers.html